数据结构 A 第 8 次作业

姓名: 王子川　　**学号:** PB25000055

下面三题中二叉树的链式存储使用如下结构体

```
typedef struct BiTNode{
    TElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode,*BiTree;
```

1(6.47) BFS

从二叉树的根 r 开始 BFS 整棵树, 用队列来实现.

```
status searchTree(BiTree T) {
    if (!T) return OK;

    LinkQueue Q;
    InitQueue(Q);
    EnQueue(Q, T);

    while (!EmptyQueue(Q)) {
        BiTree cur;
        DeQueue(Q, cur);

        visit(cur->data);

        if (cur->lchild)
            EnQueue(Q, cur->lchild);
        if (cur->rchild)
            EnQueue(Q, cur->rchild);
    }

    DestroyQueue(Q);
    return OK;
}
```

保证二叉树的每个结点都入队出队一次, 因此算法的时间复杂度为 $O(n)$, 空间复杂度为 $O(n)$, 其中 n 为树 T 的结点个数.

2(6.48) 求树中两个结点的 LCA

算法从根 T 开始递归遍历, 求出结点 p, q 的最近公共祖先 LCA.

```
BiTree LCA(BiTree T, BiTree p, BiTree q) {
    if (!T) return NULL;
    if (T == p || T == q)
        return T;

    BiTree l = LCA(T->lchild, p, q);
    BiTree r = LCA(T->rchild, p, q);
    if (l && r)
        return T;
    if (l)
```

```
        return l;
    else return r;
}
```

假设 p, q 均为二叉树中的结点. 算法的正确性证明如下. 若当前结点 T 为 p 或 q, 则它是当前子树中两结点的公共祖先, 直接返回. 否则, 若 p, q 分别在 T 的左右两个子树中, 那么 T 就是它们的 LCA. 若 p, q 都在某一个子树中, 则递归返回该子树中的 LCA. 因此算法正确.

算法的时间复杂度为 $O(n)$, 空间复杂度为 $O(h)$, 其中 n 为树 T 的结点个数, h 为树的高度.

3(6.54) 从顺序表建完全二叉树

顺序表第 i 个结点的父亲结点为 $\lfloor \frac{i}{2} \rfloor$, 左孩子的下标为 $2i$, 右孩子的下标为 $2i+1$.

```
status buildBiTree(BiTree& T, SqList sa) {
    int n = sa.last;
    if (!n) {
        T = NULL;
        return OK;
    }

    BiTree* nodes = new BiTree[n + 1];

    for (int i = 1; i <= n; i++) {
        nodes[i] = new BiTNode;
        nodes[i]->data = sa.elem[i];
        nodes[i]->lchild = nullptr;
        nodes[i]->rchild = nullptr;
    }

    for (int i = 1; i <= n; i++) {
        if (2 * i <= n)
            nodes[i]->lchild = nodes[2 * i];

        if (2 * i + 1 <= n)
            nodes[i]->rchild = nodes[2 * i + 1];
    }

    T = nodes[1];
    delete[] nodes;

    return OK;
}
```

算法首先创建全部结点, 再根据完全二叉树的顺序存储性质连接左右孩子. 时间复杂度为 $O(n)$, 辅助空间复杂度为 $O(n)$, 其中 $n = \texttt{sa.last}$.